SWE5006 Practice Project · Graduate Certificate in Designing Modern Software Systems

Speaker Script and Live Demo Run Book

BlocBuy — Team 8 · Monday 26 October 2026, 16:30–17:10 SGT (Zoom)

Responsible	Yu Qianshuo (Yorik), presentation lead and storyline (#464); Lim Jiechao, slide master.
Specification	`docs/swe5006/presentation/RUBRIC.md`: every slide records the rubric rows it covers; the coverage tables in the run sheet are derived from them.
Companion files	`swe5006-presentation.html` — the deck (36 slides + backups BX, B1–B12; Chrome or Edge; **S** notes, **T** timer, **O** overview, **F** fullscreen, **B** backup index) · `swe5006-presentation.pdf` — the same deck for submission.
Facts as of	21 Oct 2026. Dev health `UP`; #223 and #517 open, so no payment has been confirmed and no refund proven on dev yet.

Markers

`[FILL]` value that does not exist yet; never present a guess · `[VERIFY]` claim to check against the repository, report or run logs · `[CONFIRM]` needs team agreement. They must all be gone after the 25 Oct rehearsal.

Run sheet and coverage check

Slide-by-slide run sheet

Clock	Slot	Slide	Title	Speaker	Item	Est.
16:30	5 s	1	BlocBuy	Lim	1a	4 s
16:30:05	25 s	2	Group buying fails when buyers cannot commit together	Lim	1a	24 s
16:30:30	30 s	3	Scope: nine use cases, six quality attributes with targets	Lim	1b	30 s
16:31	30 s	4	Promised, delivered, changed, dropped: status on 21 October	Lim	1c	29 s
16:31:30	15 s	5	Milestones achieved and effort per member	Lim	1d	13 s
16:31:45	15 s	6	Ownership: one use case and one design problem each	Lim	1d	13 s
16:32	45 s	7	Four sprints: the Definition of Done reached, and what each retro changed	Lim	1e	42 s
16:32:45	15 s	8	Agile artefacts: a story card, a burndown, a practice	Lim	1g	14 s
16:33	45 s	9	Logical architecture: COLA layers across six bounded contexts	Eric	2a	0 s
16:33:45	40 s	10	Transition strategies from analysis to design	Eric	2b	0 s
16:34:25	35 s	11	One strategy pair: analysis objects become design classes	Eric	2b	0 s
16:35	30 s	12	Use case diagram: in scope, out of scope and status	Eric	2c	0 s
16:35:30	20 s	13	UC4 join: the analysis model	Eric	2d	0 s
16:35:50	30 s	14	UC4 join: the design model	Eric	2d	0 s
16:36:20	10 s	15	Team-formation statechart	Eric	2d	0 s
16:36:30	20 s	16	UC3 start a team: Factory Method	Eric	2e	0 s
16:36:50	20 s	17	UC5 to UC6 team outcome: Observer	Yorik	2e	0 s
16:37:10	20 s	18	Notifications: Template Method for outbox transports	Pan	2e	0 s
16:37:30	20 s	19	UC1 pricing: Strategy	Lim	2e	0 s
16:37:50	30 s	20	Design problem: listing rules that depend on its state	Liu Jin	2e	0 s
16:38:20	25 s	21	After: the State pattern, and why the team uses a table	Liu Jin	2e	0 s
16:38:45	15 s	22	Database schema: core entities and mapping decisions	Liu Jin	2f	0 s
16:39	110 s	23	Testing on the Agile Testing Quadrants	Liu Jin	3a	93 s
16:40:50	70 s	24	Scripted performance test: tool, scenario, target	Liu Jin	3a	65 s
16:42	60 s	25	Source code and artefacts	Pan	4a	0 s
16:43	60 s	26	Continuous integration: pull-request jobs and post-merge jobs	Pan	4b	0 s
16:44	45 s	27	Security controls by pipeline stage	Pan	4d	0 s
16:44:45	60 s	28	Delivery: environments, approval and verification	Pan	4b	0 s
16:45:45	45 s	29	Technical findings and issues encountered	Pan	4c	0 s
16:46:30	330 s	30	Live: one change from pull request to dev	Pan	4b	0 s

16:52	30 s	31	System demo: one bloc from start to refund	Yorik	5a	0 s
16:52:30	510 s	32	Live: two buyers, one bloc	Yorik	5a	0 s
17:01	120 s	33	Performance against the target	Yorik	5b	0 s
17:03	30 s	34	Scalability: a stated trade-off	Yorik	5b	0 s
17:03:30	30 s	35	Maintainability: the pipeline you just saw	Yorik	5b	0 s
17:04	Q&A	36	Thank you	Yorik	5b	

Segment talk budget vs slots vs estimated speaking time: §1 180 s / 180 s / 169 s · §2 360 s / 360 s / 0 s · §3 180 s / 180 s / 158 s · §4 600 s / 600 s / 0 s · §5 720 s / 720 s / 0 s.

34 minutes scripted, 16:30–17:04; the 6 minutes to 17:10 are the buffer for questions asked during the talk (rubric G1–G3). §1 runs 3 minutes and §2 6 minutes so that each member presents their own design problem (rubric G4, time budget).

Interruption policy

- A lecturer may ask at any moment. The current speaker answers in **one or two sentences** (decision → reason → evidence or limitation), says “back to …” and continues from the same slide.
- If the owner is someone else, the speaker says so and Lim posts `Q: <question> → owner` in the backchannel; parked questions are answered first in the buffer.
- Hard checkpoints: 16:33 Eric, 16:39 Liu Jin, 16:42 Pan, 16:52 Yorik.

Handovers (rubric G4: rehearse each one)

From slide	Speakers
9	Lim Jiechao → Wang Ziang (Eric)
17	Wang Ziang (Eric) → Yu Qianshuo (Yorik)
18	Yu Qianshuo (Yorik) → Panpakorn Siripanich (Pan)
19	Panpakorn Siripanich (Pan) → Lim Jiechao
20	Lim Jiechao → Liu Jin
25	Liu Jin → Panpakorn Siripanich (Pan)
31	Panpakorn Siripanich (Pan) → Yu Qianshuo (Yorik)

Every guideline item has slides and a speaker

Item	Guideline requirement (verbatim)	Slides	Speaker
1a	Explain the context and the business problem that the solution as a whole is trying to achieve.	1, 2	Lim Jiechao
1b	Explain the scope of the project.	3	Lim Jiechao
1c	Explain the current status of the project including outstanding issues.	4	Lim Jiechao
1d	Describe the actual project milestones and the total effort expended by each team member.	5, 6	Lim Jiechao
1e	Explain how many sprints planned, Definition of Done (DoD) achieved.	7	Lim Jiechao
1f	Sprint retrospective & reflection in the next sprint	—	

1g	Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.	8	Lim Jiechao
2a	Software architectural decisions that influence the logical architecture.	9	Wang Ziang
2b	Transition from analysis to design	10, 11	Wang Ziang
2c	Use case diagram	12	Wang Ziang
2d	Analysis and design models	13, 14, 15	Wang Ziang
2e	Design problems and patterns	16, 17, 18, 19, 20, 21	Wang Ziang, Yu Qianshuo, Panpakorn Siripanich, Lim Jiechao, Liu Jin
2f	Database schemas	22	Liu Jin
3a	Types of Testing Conducted	23, 24	Liu Jin
3b	Well structured source code with unit test cases for classes with more complex behaviours.	—	
4a	Describe the strategy to manage the source code and artifacts.	25	Panpakorn Siripanich
4b	Describe the DevSecOps pipeline and procedures.	26, 28, 30	Panpakorn Siripanich
4c	Highlight any technical findings and issues encountered.	29	Panpakorn Siripanich
4d	Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.	27	Panpakorn Siripanich
4e	Backup: Record a video in advance in case of any technical issues during the live demo.	—	
5a	Demonstrate some of the significant use cases of the solution.	31, 32	Yu Qianshuo
5b	Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).	33, 34, 35, 36	Yu Qianshuo
5c	Backup: Record a video in advance in case of any technical issues during the live demo.	—	

Every rubric row has a slide (rubric checklist)

Rubric row	Slides	Speaker
1.1	1, 2	Lim Jiechao
1.2	3	Lim Jiechao
1.3	4	Lim Jiechao
1.4	5	Lim Jiechao
1.5	6	Lim Jiechao
1.6	7	Lim Jiechao
1.7	7	Lim Jiechao
1.8	8	Lim Jiechao
2.1	9	Wang Ziang
2.2	10, 11	Wang Ziang

2.3	12	Wang Ziang
2.4	13, 14, 15	Wang Ziang
2.5	16, 17, 18, 19, 20, 21	Wang Ziang, Yu Qianshuo, Panpakorn Siripanich, Lim Jiechao, Liu Jin
2.6	22	Liu Jin
3.1	23	Liu Jin
3.2	24	Liu Jin
3.3	23, 27	Liu Jin, Panpakorn Siripanich
4.1	30	Panpakorn Siripanich
4.2	25	Panpakorn Siripanich
4.3	26, 27	Panpakorn Siripanich
4.4	28	Panpakorn Siripanich
4.5	29	Panpakorn Siripanich
4.6	30	Panpakorn Siripanich
5.1	31, 32	Yu Qianshuo
5.2	31, 32, 34	Yu Qianshuo
5.3	33	Yu Qianshuo
5.4	35	Yu Qianshuo
5.5	32	Yu Qianshuo

Presentation tips checklist

Tip (guidelines page 2)	How this deck meets it
Summarize and highlight the significant achievements. Talk more about the significant and interesting parts of the solution and implementation. Your audience should remember the unique aspects of the solution and not similarities to other solutions.	Depth goes to what is unique: pay-first teams that refund themselves, money safety under concurrency and failure (UC4 design, statechart, design problems), and a pipeline from pull request to dev in minutes.
Manage your time well. Do not present too many slides. Remember that the audience can read the text in the slide so refrain from reading them.	36 main slides in 34 minutes, two of them live demos; slots checked at 130 wpm in the table above; sample wording adds to the slide instead of reading it; details sit in backups B1–B12.
Explain your design in top-down manner. Make sure that your audience understands the requirements, the high-level design and the high-level decisions that you make before drilling into some details.	Problem, scope and quality attributes first (slides 2–3); then architecture (9), transition strategies (10–11), use cases (12), one use case from analysis to design (13–15), design problems (16–21), schema (22).
Ensure that the contents on the slides are readable from the distance at which your audience is sitting. This is particular true for face-to-face presentation.	Rubric V1–V7: one idea per slide, diagrams cropped to the part discussed, labels checked on a projector or TV before 26 Oct.

Cue words in the team backchannel (never the Zoom chat)

Message	Meaning	Sent by
`1 MIN`	One minute of talk left in this segment	Lim
`TIME`	Stop; finish the sentence and hand over	Lim
`Q: <text> → name`	A question was parked; the named owner answers it in the buffer (17:04–17:10)	Lim

`BEHIND n / AHEAD n`	The talk is n seconds behind or ahead of the run sheet; the next speaker applies the trims	Lim
`SHARE`	Next person: start screen sharing now	Lim
`CLIP n`	Live step n failed; play the recording of step n	Pan or Yorik

1 Project Introduction & Project Management Summary (3+0 minutes)

Clock	16:30:00 – 16:33:00 (3 min talk + 0 min Q&A; talk ends 16:33:00)
Owner	Lim Jiechao
Slides	1, 2, 3, 4, 5, 6, 7, 8

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)
1a — Explain the context and the business problem that the solution as a whole is trying to achieve.
1b — Explain the scope of the project.
1c — Explain the current status of the project including outstanding issues.
1d — Describe the actual project milestones and the total effort expended by each team member.
1e — Explain how many sprints planned, Definition of Done (DoD) achieved.
1f — Sprint retrospective & reflection in the next sprint
1g — Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.

Tip
Lim opens at 16:30:00 sharp, with the deck shared and the notes window off the shared screen. Eight slides in three minutes: the tables carry the facts, the wording carries only the point of each slide.

Slide 1 BlocBuy

16:30:00–16:30:05 · 5 s

Speaker Lim · covers guideline item **1a**

Say *Good afternoon. We're Team 8, and this is BlocBuy.*

≈ 9 words · ≈ 4 s at 130 wpm · slot 5 s

Cues

- 16:30:00 — Lim opens swe5006-presentation.html in Chrome, presses S for the notes window (keep it off the shared screen), clicks back into the deck window and presses T there to start the timer. In Zoom, share only the deck window.
- Backchannel: the team chat stays open. Lim repeats any question asked during the talk in one line and names who answers; the buffer is 17:04–17:10 (rubric G3).
- The roadmap line on the slide is the five sections of the lecturer's sequence; do not read it.

Hand-over *"The problem first."*

Slide 2 Group buying fails when buyers cannot commit together

16:30:05–16:30:30 · 25 s

Speaker Lim · covers guideline item **1a**

Say

Small merchants can only discount against committed volume, and a single buyer never reaches a bulk price. Every listing has two prices. You pay to start a team and share the link; friends join by paying too. If it fills in time, everyone gets the team price; if not, everyone is refunded automatically.

≈ 53 words · ≈ 24 s at 130 wpm · slot 25 s · if short of time: If over time, drop "and share the link".

Cues

- Source: proposal §4 (27 Aug): the pay-first mechanic, two prices, automatic refund on expiry, modelled on the early Pinduoduo playbook; §6.1: Stripe test mode only.
- The one-sentence problem is the slide's headline: say it, then the mechanism. What makes it technical is that money is committed before the team exists, so every later section is about money safety under concurrency and failure.

Hand-over *"Here is what we set out to build."*

Slide 3 Scope: nine use cases, six quality attributes with targets

16:30:30–16:31:00 · 30 s

Speaker Lim · covers guideline item **1b**

Say

Scope is the nine use cases of our proposal, from browsing to admin listings, plus in-app notifications for SWE5001. Out of scope stays as proposed: no merchant self-service, no email or push, no live payments. On the right are the six quality attributes we committed to, with their targets. Each is tagged with where we show it; the system demo proves the first three.

≈ 64 words · ≈ 30 s at 130 wpm · slot 30 s · if short of time: If over time, drop "from browsing to admin listings".

Cues

- In scope: proposal §6.1, UC1–UC9. Receive notifications (#215, #446) was added in Sprint 4 for SWE5001; it is off on the EC2 dev host by decision (backend ADR 0007, 11 Oct).
- Out of scope: proposal §6.2 verbatim list (merchant self-service GUI; analytics; recommendation; automatic team matching; notifications beyond in-app; native apps; live payments). We went beyond the first item for admins only: an admin console (frontend ADR 0015), shown on slide 4.
- Quality attributes and targets: proposal §6.3, quoted. "Shown in" maps each to the rubric rows that demonstrate it: 5.2 (correctness, security), 5.3 (performance, k6 on the SWE5001 ECS environment), 5.4 and §2 (maintainability), §3–§4 (code quality, auditability).

Hand-over *"How much of that did we deliver?"*

Slide 4 Promised, delivered, changed, dropped: status on 21 October

16:31:00–16:31:30 · 30 s

Speaker Lim · covers guideline item **1c**

Say *Against the proposal, every use case is in the code, and the admin console goes beyond it. The open issue is the money path on dev: Checkout returns, but the webhook never confirms, issues 517 and 223, our go/no-go on Friday. Five things changed, each with its record, and a SWE5006 production environment was dropped. The open gaps are at the bottom.*

≈ 62 words · ≈ 29 s at 130 wpm · slot 30 s · if short of time: If over time, drop "and the admin console goes beyond it".

Cues

- Use cases: #24–#32 all closed completed 19–28 Sep; UC8 shipped (#31, #179, #181), so it was not sacrificed; UC9 admin console: frontend ADR 0015, admin stacks #294–#338.
- Money path on dev: #517 (16 Oct, Stripe reports success with test card 4242, receipt stays "Waiting for payment confirmation") and #223 (Checkout, scheduling and expiry refunds on dev), both open on 21 Oct, owner Yorik. Go/no-go Fri 23 Oct 18:00 (open item 1). `[VERIFY: re-check #517 and #223 on 25 Oct]`
- Changed: team lifecycle State → DealStatus.canTransitionTo transition table (backend/docs/design/paid-group-participation.md; State is used for listings, slide 21). JWT → Sa-Token session in Redis with an HttpOnly cookie (backend-skeleton-plan.md D6/D7; frontend ADR 0002; no backend ADR, open item 7). JPA → MyBatis behind repository ports (skeleton plan §3 stack; no ADR, open item 7). ECS Fargate + RDS for dev → one EC2 host with Docker Compose and PostgreSQL on the host (backend ADR 0008). Secrets Manager → SSM SecureString and a root-only host env file (docs/deploy/backend-ec2-pipeline-operator-guide.md §7).
- Dropped for SWE5006: the prod release with a human gate and the ECS circuit breaker (#226 closed not planned 11 Oct; both exist for SWE5001 on ECS). Partial: Playwright e2e runs on API stubs, the signed-in suite on dev (#229) was not planned; burndown data is captured daily (board-snapshot.yml since 17 Sep) but no chart is generated; saved-plan Terraform promotion is a manual operator step only. Delivered: Checkstyle and both SonarCloud gates as required checks.
- Open gaps: port 22 open to 0.0.0.0/0 on the dev host (AWS read-only, 21 Oct); CodeQL off (ENABLE_CODEQL=false); CloudFront → ALB over HTTP accepted until 16 Nov (#227, NFR-37); dev EBS volume unencrypted (AWS, 19 Oct).

Hand-over *"How we got here, in milestones and hours."*

Slide 5 Milestones achieved and effort per member

16:31:30–16:31:45 · 15 s

Speaker Lim · covers guideline item **1d**

Say *Milestones left: the pipeline on dev in week one, the web app on 22 September, a tag per sprint. Right: hours per member, planned, because nobody logged actuals.*

≈ 28 words · ≈ 13 s at 130 wpm · slot 15 s · if short of time: If over time, drop "a tag per sprint".

Cues

- Milestones: proposal header (27 Aug); Sprint 1 report (dev pipeline live 5 Sep); stripe-sandbox-acceptance-20260920.md (local Sandbox acceptance 20 Sep); #93 comment (web app on dev 22 Sep); tags v0.1.0-sprint1/2/3 (14 Sep, 27 Sep, 11 Oct; git tag); docs/perf/sprint3-baseline.md (30 Sep, local); PR #424 merged 13 Oct.
- Planned hours: Sprint 2 plan docx (30 h each), sprint-03/plan.md §4 (30 h each), sprint-04/plan.md §5 (Liu Jin 51, Yorik 48, Eric 46, Pan 41.5, Lim 38). Open PR #538 rebalances Sprint 4 to Liu Jin 66, Eric 26, Pan 46.5 (open on 21 Oct). Sprint 1 hours were not recorded (Sprint 1 report contribution table left TODO). Team done hours: Sprint 2 134 of 150 (Sprint 2 report), Sprint 3 117 of 150 (sprint-03/review.md).
- Rubric 1.4: effort figures must match the fortnightly progress reports already submitted. Those reports carry no hours per member (Sprint 1 TODO, Sprint 2 activities and PR counts), so never present the planned hours as actuals.
  `[CONFIRM: actual effort per member agreed by all five, reconciled with the submitted progress reports]`

Hand-over *"Who owned what."*

Slide 6 Ownership: one use case and one design problem each

16:31:45–16:32:00 · 15 s

Speaker Lim · covers guideline item **1d**

Say *Each of us owns one use case and one design problem, and presents that problem in the next section. The last column shows what moved from the proposal's allocation.*

≈ 29 words · ≈ 13 s at 130 wpm · slot 15 s

Cues

- Ownership: issue #463 and sprint-04/plan.md §11 for the use cases; design problems as decided on 21 Oct (Strategy for UC1 pricing assigned to Lim, Template Method to Pan, Observer to Yorik). Proposal §6.1 indicative allocation: UC1 Strategy Pan; UC2 Pan; UC3 Factory Method and UC4 Eric; UC5 State/Observer and UC9 Liu Jin; UC6 Adapter Yorik; UC7 Observer and UC8 Jiechao.
- Also on the matrix: the team-formation statechart (UC4–UC6, slide 15) is presented by Eric; the code behind it (DealStatus, GroupDeal guards) is in his UC3/UC4 design.
  `[VERIFY: UC9 attribution: #32 and backend design docs name Eric as UC9 implementer (PR #72); #463 gives UC9 to Liu Jin (open item 12)]`
- Report sections: RT 3.4.N (use case by member) and 3.5.N (design problem by member).
  `[FILL: report section numbers once #463 fixes the chapter order]`

Hand-over *"And how each sprint went."*

Slide 7 Four sprints: the Definition of Done reached, and what each retro changed 16:32:00–16:32:45 · 45 s

Speaker Lim · covers guideline item **1e**

Say *We place each sprint on the Definition of Done ladder from the EPAT module: coded, tested, integrated, releasable, released. Sprint 1 put the backend on dev, but no frontend. Sprint 2 deployed both halves, but with Checkout off the money path stayed tested, not releasable. Sprint 3 aimed for a refund on dev; our rule says a local run does not count, so: integrated.* **[click]** *Each retro changed the next sprint: no burndown led to daily board snapshots; a ten-thousand-line pull request led to a 1,500-line limit and a mid-sprint check.*

≈ 90 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, drop "but no frontend" and "a mid-sprint check".

Cues

- Ladder: EPAT 01 Agile p26 (coded → tested → integrated → releasable → released). Placing each sprint on it is our own reading of the evidence, not a course rating.
- Sprint 1 (2–12 Sep, #49): backend pipeline live on dev 5 Sep with health check and rollback; frontend not started (Sprint 1 report). Sprint 2 (14–25 Sep, #93): 9 of 11 stories, 134 of 150 h; frontend live 22 Sep; Checkout and scheduling off on dev (Sprint 2 report). Sprint 3 (28 Sep–9 Oct, #232): 117 of 150 h; review records the goal as not met on dev (sprint-03/review.md); DoD rule "on dev means demonstrated on deployed dev" (sprint-03/plan.md §8). Sprint 4 (12–23 Oct, #465):
  `[FILL: DoD step as of the 25 Oct rehearsal, after the 23 Oct review]`
- Retro → change: Sprint 1 retro (Sprint 1 report: no burndown or story points) → dated day logs and the scheduled board snapshot to docs/agile/metrics/ from 17 Sep (Sprint 2 report, "Acting on the Sprint 1 retrospective"). Sprint 2 retro (Sprint 2 report: ≈ 10,000-line Stripe slice, cliff burndown) → sprint-03/plan.md §8: PRs under ~1,500 lines, remaining hours in the estimate field, mid-sprint check 2 Oct. Sprint 3 retro not written (#451): the only recorded action is sprint-04/plan.md §7 (owners update the estimate field; the mid-sprint check verifies it). Sprint 4 retro due 23 Oct.
  `[FILL: Sprint 3 and Sprint 4 retro rows (open items 5, 6)]`
- One or two improvements per sprint follows the EPAT retrospective template (SCRUM_Process_Team01.xlsx, "Focus on 1 - 2 improvements").

Hand-over *"Two artefacts from that process."*

Slide 8 Agile artefacts: a story card, a burndown, a practice

16:32:45–16:33:00 · 15 s

Speaker Lim · covers guideline item **1g**

Say *From our GitHub board: one story card with its acceptance criteria, Sprint 2's burndown, flat then a cliff, and a refactoring with test-first evidence. The full set is in the appendix.*

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s

Cues

- Story card: #221 UC9-ext (US25), owner Liu Jin, six acceptance criteria ticked, merged in PR #242 (29 Sep), design problem field names State and Optimistic Offline Lock.
- Burndown: docs/sprint2-progress-report.typ, "Sprint Burndown Chart" (reconstructed from merge dates because the estimate field was empty); daily data docs/agile/metrics/*.csv since 17 Sep.
- Practice (EPAT 05 refactoring, EPAT 03 TDD): two red tests reproduced the below-one-count quote defect before the fix (docs/agile/sprint-03/log/2026-10-07.md, GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted), then PR #424 (13 Oct) refactored admission into shared guards and a constructor-injected Factory Method (#421, #422).
- Full set (rubric 1.8: product backlog, user stories, sprint plans, burndowns, reviews, retros): backup B1, burndowns B2.

Hand-over *"Eric takes you through the design."*

Watch out

Never present planned hours as actual effort (slide 5), and never say payments or refunds work on dev unless the 24 Oct dry run proved it (slide 4). `[CONFIRM: slide 4 wording after the 23 Oct go/no-go]`

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why is Sprint 3 only "integrated" when the code was merged?	Our Definition of Done says "on dev" means shown on the deployed environment; a local run does not count. Checkout and scheduling were still off on dev at the freeze (#223), so the review recorded the goal as not met and carried it into Sprint 4. Evidence: sprint-03/review.md, sprint-03/plan.md §8.	Lim
Are these hours actuals?	No. We planned in hours per story; done hours are planned hours of finished work (Sprint 2 report, Sprint 3 review), and the Sprint 4 burndown sums planned estimates of cards not yet Done. Actual hours per person were not logged; that is a limitation we state.	Lim
Was the effort balanced across the five members?	The plans gave everyone 30 h in Sprints 2 and 3; Sprint 4 started at 38–51 h each and is being rebalanced (PR #538). Board cards differ in size, for example one Stripe pull request was about 10,000 lines, so card counts are not effort. `[CONFIRM: the effort split all five agree on]`	Lim
The proposal promised JWT. Where is that change recorded?	In the locked backend decisions of 30 Aug (backend-skeleton-plan.md D6/D7: Sa-Token session in Redis, RBAC from SQL tables) and frontend ADR 0002 (same-origin /api, HttpOnly cookie). Reason: a server session can be revoked and keeps the token out of JavaScript. Limitation: there is no numbered backend ADR for it (open item 7).	Lim → Liu Jin
Why did you build an admin GUI when the proposal excluded it?	The exclusion was a merchant self-service GUI. Admins needed to configure team terms on real listings to make START possible on dev (US25, #221), and an API-only flow was error-prone in the demo; the console reuses the same versioned API (frontend ADR 0015). It is extra scope, not a replacement.	Lim
What is your biggest open risk for the demo?	The Stripe webhook on dev (#517): payments stay waiting, so paid START, JOIN and refunds do not complete on dev yet. Go/no-go for the live payment steps is 23 Oct, with backup recordings by 24 Oct.	Lim → Yorik

2 Analysis & Design and Design Patterns (6+0 minutes)

Clock	16:33:00 – 16:39:00 (6 min talk + 0 min Q&A; talk ends 16:39:00)
Owner	Wang Ziang (Eric) for 2.1–2.4 and his design problem; then Yu Qianshuo (Yorik), Panpakorn Siripanich (Pan) and Lim Jiechao, each on their own design problem; Liu Jin for the full-form problem and 2.6
Slides	9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

2a — Software architectural decisions that influence the logical architecture.
2b — Transition from analysis to design
2c — Use case diagram
2d — Analysis and design models
2e — Design problems and patterns
2f — Database schemas

Tip

Eric presents 2.1–2.4 and his own design problem; then Yorik, Pan and Lim each have one 20-second slide on their own problem; Liu Jin closes with the full-form State problem and the schema, and carries straight on into §3. The next speaker unmutes during the previous speaker's last sentence.

Slide 9 Logical architecture: COLA layers across six bounded contexts

16:33:00–16:33:45 · 45 s

Speaker Eric · covers guideline item **2a**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 45 s

Slide 10 Transition strategies from analysis to design

16:33:45–16:34:25 · 40 s

Speaker Eric · covers guideline item **2b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 40 s

Slide 11 One strategy pair: analysis objects become design classes

16:34:25–16:35:00 · 35 s

Speaker Eric · covers guideline item **2b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 35 s

Slide 12 Use case diagram: in scope, out of scope and status

16:35:00–16:35:30 · 30 s

Speaker Eric · covers guideline item **2c**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 30 s

Slide 13 UC4 join: the analysis model

16:35:30–16:35:50 · 20 s

Speaker Eric · covers guideline item **2d**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 20 s

Slide 14 UC4 join: the design model

16:35:50–16:36:20 · 30 s

Speaker Eric · covers guideline item **2d**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 30 s

Slide 15 Team-formation statechart

16:36:20–16:36:30 · 10 s

Speaker Eric · covers guideline item **2d**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 10 s

Slide 16 UC3 start a team: Factory Method

16:36:30–16:36:50 · 20 s

Speaker Eric · covers guideline item **2e**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 20 s

Slide 17 UC5 to UC6 team outcome: Observer

16:36:50–16:37:10 · 20 s

Speaker Yorik · covers guideline item **2e**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 20 s

Slide 18 Notifications: Template Method for outbox transports

16:37:10–16:37:30 · 20 s

Speaker Pan · covers guideline item **2e**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 20 s

Slide 19 UC1 pricing: Strategy

16:37:30–16:37:50 · 20 s

Speaker Lim · covers guideline item **2e**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 20 s

Slide 20 Design problem: listing rules that depend on its state

16:37:50–16:38:20 · 30 s

Speaker Liu Jin · covers guideline item **2e**

Say [FILL: draft pending]

≈ 1 words · ≈ 0 s at 130 wpm · slot 30 s

Slide 21 After: the State pattern, and why the team uses a table

16:38:20–16:38:45 · 25 s

Speaker Liu Jin · covers guideline item **2e**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 25 s

Slide 22 Database schema: core entities and mapping decisions

16:38:45–16:39:00 · 15 s

Speaker Liu Jin · covers guideline item **2f**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 15 s

Watch out

Segment 2 has no slack: answer design questions in one sentence and park the rest.

`[FILL: segment 2 slides pending (slides 9–22)]`

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why a monolith on dev and services on prod?	Decision: one codebase, two topologies (backend ADR 0008). Reason: SWE5006 grades a complete runnable system, so dev stays the monolith on one EC2 host; SWE5001 needs three services on ECS. The services are thin assemblies of the same modules, and ArchUnit holds the boundaries in the monolith too.	Eric
Is ProductStatus really the State pattern, or an enum switch?	State, with enum constants as the concrete states: each constant with a body compiles to its own subclass, overrides only the moves it allows and returns the next state; the defaults refuse with 409, and Product delegates without branching on status. Evidence: ProductStatus.java, ProductStatusTest. GroupDeal uses a transition table (DealStatus.canTransitionTo), which we do not call State.	Liu Jin
How do you stop a double refund?	A durable obligation, then an idempotent call. The obligation is unique per payment and commits with the expiry; the Stripe idempotency key is built from its ID; a lease plus a version fence stops two workers; after 23 hours the worker only searches Stripe, never creates. Evidence: RefundServiceTest.lostResponseRetriesSameDurableKey. Limitation: proven locally with a provider substitute, not on dev (#223).	Yorik
Why a fixed 30 s retry and not exponential backoff?	A fixed delay in RefundMapper.xml: the idempotency key makes each retry safe, and the 23-hour cutoff ends automatic retries. There is no exponential backoff or jitter, so we call it a fixed delay.	Yorik
Why a Factory Method with only one creator?	The creator is a seam: initiate is final, so any creator keeps the validation, and constructor injection (#421) lets a test or configuration supply another creator without editing the service. Limitation: one creator, chosen at build time; we do not claim runtime-pluggable creation.	Eric
Is there a database per service?	On dev: one PostgreSQL database, schema "domain", all tables. On ECS: one RDS instance with a database per service, and each runtime role is granted only its own tables (ADR 0008). Full list: B4.	Liu Jin
Do notifications work on dev?	No, by decision: the dev host runs no broker and leaves the outbox writer, relay and consumer off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). The path runs on ECS through SNS and SQS, and locally and in CI against ElasticMQ (OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT).	Pan

3 Types of Testing Conducted (3+0 minutes)

Clock	16:39:00 – 16:42:00 (3 min talk + 0 min Q&A; talk ends 16:42:00)
Owner	Liu Jin
Slides	23, 24

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

3a — Types of Testing Conducted
3b — Well structured source code with unit test cases for classes with more complex behaviours.

Tip

Liu Jin continues from slide 22 without a handover.

Slide 23 Testing on the Agile Testing Quadrants

16:39:00–16:40:50 · 110 s

Speaker Liu Jin · covers guideline item **3a**

Say

We map our tests onto the Agile Testing Quadrants and tag each with its level. Quadrant one, technology-facing and supporting the team: 185 unit test classes in JUnit and Mockito, 77 Vitest component files and 18 ArchUnit rules, all in the required build on every pull request. They target the complex classes we promised: the team state machine; capacity at the last seat, where two threads race on real PostgreSQL and exactly one wins; and refund idempotency, where a lost Stripe response is retried with the same key. Two red tests caught a join-quote defect before its fix. **[click]** *Quadrant two, business-facing: 49 integration classes, 45 on real PostgreSQL, Redis and ElasticMQ in Testcontainers, a contract check, and eight Playwright specs, run on the preview build and again against dev after each deploy. That smoke test caught a 502 while the single host restarted.* **[click]** *Quadrant three is manual, and we keep it honest: a logged Stripe Sandbox acceptance run, locally, and sign-in cookie checks on dev; no end-user acceptance testing.* **[click]** *Quadrant four uses tools: the load test on the next slide, and four kinds of security scanning, static analysis, dependencies, image and infrastructure configuration, and secrets, which Pan shows gating the pipeline.*

≈ 201 words · ≈ 93 s at 130 wpm · slot 110 s · if short of time: If over time, drop "Two red tests caught a join-quote defect before its fix." (it stays on the slide).

Cues

- Model: EPAT 03 Test Driven Development v2.1 p7 (Agile Testing Quadrants: Q1 unit and component, automated; Q2 functional and story tests, automated and manual; Q3 exploratory, usability, UAT, manual; Q4 performance, load, security, tools) and p5 (levels: unit, integration, system). The level tag on each row is our classification.
- Counted in the repository on 22 Oct at a1c6ac6e: find backend -name "*Test.java" = 186 (185 + ArchitectureTest), 1,329 @Test/@ParameterizedTest; "*IT.java" = 49, 45 of them start Testcontainers (postgres:18, redis:8.10, ElasticMQ), 242 annotations; grep -c @ArchTest ArchitectureTest.java = 18; frontend/src "*.test.ts*" = 77; frontend/e2e = 8 specs, 24 test() declarations. Per module: backup B7.
- Complex classes (proposal §6.3 "unit tests concentrated on classes with complex behaviour"): state machine DealStatusTest, PaidGroupDealTest (terminalStatesRejectJoins, lastPaidMemberSucceedsWithoutMutatingOriginalSnapshot); capacity StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat (two threads, real PostgreSQL and MyBatis, provider doubled; asserts 1 order, 1 refund obligation, 2 succeeded payments; a *Test by name, integration by level); refund idempotency RefundServiceTest.lostResponseRetriesSameDurableKey.
- Caught: (Q1) sprint-03/log/2026-10-07.md, two zero/negative-count red tests reproduced the quote defect (GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted; #422, fixed in #424); ArchUnit controllers_are_validated required @Validated on InternalUserController (sprint-04/log/2026-10-13.md, #468). (Q2) ci-frontend runs 37102432667 (3 Oct) and 37182996807 (4 Oct): e2e-dev got 502 from /api/actuator/health inside deploy-existing-ec2 of the same commit.
- Where it runs: ci-backend.yml build-quality (mvnw verify: unit, ArchUnit, ITs; required), contract-check (export, git diff --exit-code contracts/, openapi-diff, not required); ci-frontend.yml frontend-check (Vitest; required), e2e-preview (PRs), e2e-dev (after deploy-s3-dev). Latest local full backend verify: 2,048 passed, 1 opt-in performance skip (sprint-04/log/2026-10-21.md, base 30b75aaa).
- Q3 evidence: backend/docs/payment/stripe-sandbox-acceptance-20260920.md (browser START and JOIN through Hosted Checkout, local, 20 Sep); sprint-03 8 Oct log: Google sign-in on dev, Chrome DevTools shows blocbuy-token Secure and HttpOnly;

hand mutation probes 12–13 Oct logs. No user acceptance testing with outside users; say so if asked.
- Q4 security: types only here (rubric 3.3). Tools and gates are on slide 27: SAST SonarCloud + SpotBugs (CodeQL off), dependencies Trivy fs + Dependabot, image Trivy, IaC configuration Trivy config, secrets gitleaks.

Hand-over *"And this is the test that answers the performance target."*

Slide 24 Scripted performance test: tool, scenario, target

16:40:50–16:42:00 · 70 s

Speaker Liu Jin · covers guideline item **3a**

Say

Our scripted performance test is k6, pinned to one version and image digest. It runs against the SWE5001 ECS environment, which runs the same code; we never load the shared dev host, which also carries its database. The scenario steps the arrival rate from 10 to 200 requests a second, one minute per step, seven listing pages to three details, with no think time. Each step must keep p95 under 500 milliseconds, errors under 1 percent, and drop no scheduled requests. The proposal's target is 2 seconds at 100 concurrent users. By Little's law, 100 users who think for half a second send about 190 requests a second, so the top step covers the target with a gate four times stricter. Expiry within a minute was measured locally: 30 teams, worst lag 58 seconds. Yorik runs this in the system demo.

≈ 141 words · ≈ 65 s at 130 wpm · slot 70 s · if short of time: If over time, drop "pinned to one version and image digest".

Cues

- Tool and scenario: perf/k6/README.md §V2 and perf/k6/browse.js: grafana/k6:1.6.1 pinned by digest; ramping-arrival executor, 10, 20, 40, 60, 80, 100, 125, 150, 175, 200 iterations/s, 60 s each; one request per iteration, 7 list : 3 detail; no sleep; 100 preallocated VUs, 400 max; 10 s request timeout; gates per plateau p95 < 500 ms, error rate < 1%, zero dropped iterations (calibrate.mjs). Anonymous browse only: authenticated load needs real sessions.
- Environment: SWE5001 ECS, CloudFront d3lvq02o2e7tcn.cloudfront.net → ALB → ECS Fargate; the README says "Do not load-test shared SWE5006 dev". Say "SWE5001 environment, same code" every time; never call it a dev result.
- Target: proposal §6.3 "Interactive operations respond within 2 seconds under demonstration load (up to 100 concurrent users); team expiry is resolved within 1 minute of window lapse." Little's law N = X(R + Z): N = 100, Z = 0.5 s think, R ≈ 0.03 s gives X ≈ 189 requests/s. The 0.5 s think time is our assumption; with longer think times the offered load is lower.
- History, not the 5.3 result: SWE5001 V2 baseline 16 Oct (docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md), one monolith task, two runs of 57,600 requests, zero errors, every plateau p95 ≤ 57 ms; saturation not observed, so 200 requests/s is a tested lower bound. Local Node workload 30 Sep (docs/perf/sprint3-baseline.md): 100 closed-loop VUs, 200 ms think, p95 browse 24 ms, JOIN intent 57 ms, My Orders 18 ms; 30 expiries, worst observation lag 58.18 s. Local only.
- Expiry on ECS: the V4 burst (2,000 teams, perf/k6/SCALING.md) has not been run; `[FILL: V4 expiry result on SWE5001 ECS, or say "local only"]`. The 5.3 run and its report: `[FILL: pre-run date, run label and summary path]`.

Hand-over *"Pan will now show where these tests gate the pipeline."*

Watch out

Never present a local or SWE5001 result as a SWE5006 dev result: say where each number ran.
`[FILL: segment 3 slides pending (slides 23-24)]`

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How do you test payments without calling Stripe?	Stripe sits behind gateway ports (Checkout and Refund gateways). Unit tests swap the port; StripeRolloutAcceptanceIT (8 tests) runs the real HTTP endpoints, scheduler, Flyway schema and webhook signature checks, with only OAuth and the Stripe network calls substituted. Limitation: no live Sandbox refund on dev yet (#223, #517).	Liu Jin → Yorik
Why Testcontainers rather than H2?	Real PostgreSQL 18, Redis 8.10 and ElasticMQ in containers. Our invariants live in the database: UNIQUE keys, ON CONFLICT DO NOTHING, guarded UPDATEs, role grants; H2 would accept or reject different SQL. Evidence: OrderRepositoryIT.duplicatePaymentIsRejectedAtTheDatabase. Cost: Docker is needed and the backend build takes about 13 minutes.	Liu Jin
Do you do TDD?	Not for everything, and we say so. Test-first for selected changes: the rate-limit checks failed (429 expected, 200 actual) before the code (3 Oct log); two red tests reproduced the join-quote defect before the fix (7 Oct log). Elsewhere tests come in the same pull request, and the 80% new-code gate makes them mandatory.	Liu Jin
What is your test coverage?	SonarCloud on dev, read 19 Oct: backend 99.4% overall and 99.2% on new code; frontend 96.1% overall and 96.2% on new code. The gate is at least 80% on new code (NFR-06). Limitation: coverage is not assertion quality, and we have no automated mutation testing. `[VERIFY: re-read SonarCloud on 25 Oct]`	Liu Jin
How do you test the frontend against the API?	Contract first. The backend regenerates contracts/openapi.json and fails on drift; openapi-diff fails a breaking change unless labelled. The frontend generates its client and its MSW handlers from that file (frontend ADRs 0003, 0011), and an unhandled request fails a test. Limitation: stubs prove the frontend half only.	Liu Jin → Lim
Why is the performance test not on dev?	Dev users exist only through Google sign-in, so 100 authenticated sessions cannot be scripted there, and dev is one host shared with its database. The scripted k6 test runs on the SWE5001 ECS environment, the same code (slide 24, run in §5); the Node workload ran locally on 30 Sep (docs/perf/sprint3-baseline.md). Neither is a dev result, and each is labelled.	Liu Jin
Is 200 requests a second the same as 100 concurrent users?	Not directly, so we convert. k6 drives an open arrival rate; the proposal counts users. Little's law N = X(R + Z): 100 users thinking 0.5 s with about 30 ms responses send about 190 requests/s, inside the 200 we drive, and our gate is p95 < 500 ms, four times stricter than 2 s. The 0.5 s think time is our assumption. The local closed-loop run (30 Sep) used exactly 100 users with 200 ms think: p95 at most 57 ms.	Liu Jin
Is the last-seat race test a unit test?	By name only. StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat runs two threads against real PostgreSQL and MyBatis with the payment provider doubled, so by level it is an integration test. The pure unit tests of the same rule are PaidGroupDealTest (the aggregate's guards) and DealStatusTest (the transition table).	Liu Jin
Why no end-user acceptance testing?	Our quadrant three is manual checks by the team, logged: the Stripe Sandbox browser acceptance (20 Sep, local) and sign-in cookie checks on dev (8 Oct). We had no access to outside users within the course, and we do not claim UAT.	Liu Jin

4 CI/CD Pipeline Demo (10+0 minutes)

Clock	16:42:00 – 16:52:00 (10 min talk + 0 min Q&A; talk ends 16:52:00)
Owner	Panpakorn Siripanich (Pan)
Slides	25, 26, 27, 28, 29, 30

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

4a — Describe the strategy to manage the source code and artifacts.
4b — Describe the DevSecOps pipeline and procedures.
4c — Highlight any technical findings and issues encountered.
4d — Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.
4e — Backup: Record a video in advance in case of any technical issues during the live demo.

Tip
Slides 25–29 take 270 s; the live pipeline run fills slide 30 (330 s).

Slide 25 Source code and artefacts

16:42:00–16:43:00 · 60 s

Speaker Pan · covers guideline item **4a**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 60 s

Slide 26 Continuous integration: pull-request jobs and post-merge jobs

16:43:00–16:44:00 · 60 s

Speaker Pan · covers guideline item **4b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 60 s

Slide 27 Security controls by pipeline stage

16:44:00–16:44:45 · 45 s

Speaker Pan · covers guideline item **4d**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 45 s

Slide 28 Delivery: environments, approval and verification

16:44:45–16:45:45 · 60 s

Speaker Pan · covers guideline item **4b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 60 s

Slide 29 Technical findings and issues encountered

16:45:45–16:46:30 · 45 s

Speaker Pan · covers guideline item **4c**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 45 s

Slide 30 Live: one change from pull request to dev

16:46:30–16:52:00 · 330 s

Speaker Pan · covers guideline item **4b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 330 s

Watch out

`[FILL: pipeline run book (steps, preconditions, screen layout, fallback clips) pending]`

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why EC2 and not ECS for dev?	Dev reuses one existing EC2 host with the monolith in Docker Compose (operator guide; backend ADR 0008): SWE5006 needs one complete, always-on system at low cost, and the monolith fits one t3.large. Limitation: one host, so a deploy restarts the API (502 on 3–4 Oct), and PostgreSQL and Redis share the host outside the pipeline.	Pan
How do you roll back?	Automatic: deploy.sh health-checks the new container and restarts the previous digest if it fails; the workflow also checks public health. Manual: rollback-backend.yml (reason, confirmation, same concurrency group). Limitation: rollback-backend.yml has never run and no automatic rollback has fired. Flyway is never reversed, so migrations stay backward-compatible.	Pan
How are secrets handled?	CI holds no AWS keys: OIDC gives short-lived roles per job. The repository has three Actions secrets, none for AWS or the app (SONAR_TOKEN, TYPESAFE_API_KEY, BOARD_SNAPSHOT_TOKEN; gh secret list, 19 Oct). Dev application secrets live in /etc/blocbuy/backend.env (root, 0600). Limitation: the account still has two IAM users for SSM port forwarding, one with an active key (governance README).	Pan
Why is CodeQL off?	The job exists in security.yml behind ENABLE_CODEQL, which is false: code scanning is not enabled for this private repository (GitHub API, 19 Oct). SAST is covered by SonarCloud on every PR plus SpotBugs in the build. We list it as a gap (B9).	Pan
How long from merge to live?	Medians from gh on 19 Oct: backend about 3.1 minutes from the push run starting to the deploy job ending (88 runs); frontend about 2.7 minutes including the e2e-dev smoke (94 runs). PR checks take about 16 minutes for backend and 5 for frontend. Per-workflow table: B8.	Pan
How do you manage dependencies?	Dependabot checks Maven, npm, Docker and GitHub Actions daily with a 7-day cooldown; majors of Spring, Java, React and TypeScript are ignored on purpose. Every update passes the same required checks. Trivy scans dependencies on change and weekly. Actions are pinned to commit SHAs.	Pan → Lim

5 System Demo (12+6 minutes)

Clock	16:52:00 – 17:10:00 (12 min talk + 6 min Q&A; talk ends 17:04:00)
Owner	Yu Qianshuo (Yorik); Wang Ziang (Eric) drives the joiner's browser
Slides	31, 32, 33, 34, 35, 36

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

5a — Demonstrate some of the significant use cases of the solution.
5b — Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).
5c — Backup: Record a video in advance in case of any technical issues during the live demo.

Tip

Slide 31 opens the demo; slide 32 is the live two-user run (510 s); slides 33–35 cover performance, scalability and maintainability.

Slide 31 System demo: one bloc from start to refund

16:52:00–16:52:30 · 30 s

Speaker Yorik · covers guideline item **5a**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 30 s

Slide 32 Live: two buyers, one bloc

16:52:30–17:01:00 · 510 s

Speaker Yorik · covers guideline item **5a**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 510 s

Slide 33 Performance against the target

17:01:00–17:03:00 · 120 s

Speaker Yorik · covers guideline item **5b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 120 s

Slide 34 Scalability: a stated trade-off

17:03:00–17:03:30 · 30 s

Speaker Yorik · covers guideline item **5b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 30 s

Slide 35 Maintainability: the pipeline you just saw

17:03:30–17:04:00 · 30 s

Speaker Yorik · covers guideline item **5b**

Say `[FILL: draft pending]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 30 s

Slide 36 Thank you

17:04 · Q&A

Speaker Yorik · covers guideline item **5b**

Say *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

Watch out

`[FILL: system demo run book (go/no-go table, steps, preconditions, fallback clips) pending]`

Q&A (17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why is there no auto-scaling on dev?	Dev is one EC2 host running the monolith in Docker Compose, with PostgreSQL and Redis on the host (backend ADR 0008), so SWE5006 needs one complete, always-on system at low cost. Auto-scaling exists only in the SWE5001 topology on ECS (commerce 2–6 tasks on ALB requests per target, read 21 Oct), which we do not claim for SWE5006. Limitation: the dev host is a single point of failure.	Yorik → Pan
Is the refund real?	A real Stripe Sandbox refund: the Refund API in test mode, so no money moves. Locally on 20 Sep, refund re_3UHbeV… reached succeeded and a recheck created no duplicate (B11). On dev: `[FILL: Sandbox refund id from the 24 Oct dry run, or say "not yet proven on dev" (#223, #517)]` .	Yorik
What happens if Stripe is down?	The Checkout attempt is saved before Stripe is called, so a timeout leaves one recoverable attempt with a stable provider key, never a second payment (StripeCheckoutFlowTest). Refunds stay as durable obligations; the worker retries every 30 s with the same idempotency key, and after 23 hours it only searches Stripe and hands mismatches to a person.	Yorik
How do you avoid a double payment?	A repeated Checkout request with the same buyer, intent and Idempotency-Key reuses the payment. Webhook events are stored by event ID, so a replay changes nothing (StripeLocalIntegrationTest.eightConcurrentWebhookReplaysStillProduceOneMemberAndOneOrder). When two paid buyers race for the last seat, one gets it and the other gets a refund obligation (StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat).	Yorik → Eric
Why did the demo not show notifications?	By decision: the dev host runs no message broker, so the outbox writer, relay and consumer are off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). Notifications run on SWE5001 prod through SNS and SQS, and locally and in CI against ElasticMQ (slide 18).	Pan
(Questions parked during the talk)	Lim reads each parked question from the backchannel in one line and names the owner; those come first. Answer each in 30–45 s: decision, reason, evidence or limitation.	Lim

6 Timeline before, on and after the day

Before the day

Date	Do	Owner
Wed 21 Oct	Deck re-planned from the rubric; §1 drafted.	Lim
Fri 23 Oct	**Slides due** (#464). Each owner reviews their slides, cards and Q&A rows; the Sprint 4 review and retro feed slides 5 and 7. **18:00 go/no-go** for the live payment steps, posted in #464.	All; Yorik decides with Lim
Sat 24 Oct	**Backup videos** of both demos recorded on whatever is deployed then, 1080p, one clip per step, stored in the cloud and on a USB drive (rubric G6). k6 pre-run on the SWE5001 ECS environment; keep its summary.	Pan (pipeline); Yorik, Eric (system)
Sun 25 Oct	**Timed rehearsal** in Zoom at the real clock with injected questions; every rubric checklist box ticked; every marker resolved; rebuild deck and script.	All; Lim times

Presentation day (Mon 26 Oct)

Time (SGT)	Do	Owner
≤ 12:00	Health and recent runs (A.1); `[FILL: demo preconditions]`	Yorik, Pan
14:30	Tech check in a test Zoom: mic, camera, sharing, Do Not Disturb.	All
15:00	Merge freeze on `dev` until 17:15.	Lim
16:30	Start; follow the run sheet. The scribe logs every remark verbatim (rubric G9).	Lim; scribe
17:04 / 17:10	Slide 36; parked questions; end.	Lim

After the presentation

- The same evening: every lecturer remark goes into #463 as a report task (by 27 Oct, rubric G9).
- Day log under `docs/agile/sprint-04/log/`: the demo PR, run links and any payment or refund ids shown (no secrets).

7 Open items before the 25 Oct rehearsal

There are 36 `[FILL]`, 3 `[VERIFY]` and 3 `[CONFIRM]` markers in this script. **FILL** is a value that does not exist yet, so never present a guess; **VERIFY** is a claim to check against the repository, the report or run logs; **CONFIRM** needs team agreement. All must be gone after the 25 Oct rehearsal.

#	Item	Owner	Blocks slide
1	Fix the Stripe webhook on dev (#517) and enable Checkout, scheduling and expiry refunds (#223); go/no-go for the live payment steps on Fri 23 Oct 18:00	Yorik	4, 32
2	Backup videos of both demos recorded by Sat 24 Oct on whatever is deployed then, stored in the cloud and on a USB drive (rubric G6, 4.6, 5.5)	Pan (pipeline), Yorik and Eric (system)	30, 32
3	Actual effort per member, agreed by all five and reconciled with the submitted progress reports (rubric 1.4); never show planned hours as actuals	Lim	5
4	Sprint 4 planned hours per member once PR #538 merges (Liu Jin 66, Eric 26, Pan 46.5)	Eric	5
5	Sprint 3 retrospective (#451): the Sprint 3 → 4 row of slide 7	Lim	7
6	Sprint 4 result, DoD step reached, retrospective and the v0.1.0-sprint4 tag date after the 23 Oct review, stated as of the rehearsal	Eric	5, 7
7	No numbered backend ADR for: the auth change from JWT to the Sa-Token session (open decision 6); MyBatis instead of the proposal's JPA; the scheduling design; the concurrency approach (open decision 3). Write them up, or cite the D-table and design docs and state the gap	Liu Jin, Eric	4, 10
8	Name a scribe for 26 Oct who logs every lecturer question and remark verbatim; turn the log into #463 tasks by 27 Oct (rubric G9)	Lim	—
9	Dev host security group allows SSH (22) from 0.0.0.0/0 although SSM sessions work (AWS read-only, 21 Oct): close it or record why it stays open	Pan	4, 28
10	Questions to Chandra: is Q&A inside the 40 min; is there a marking rubric; is the presentation graded individually (open decision 9); report due 1 Nov (briefing) or 10 Nov (#463)	Lim	—
11	Re-check slide 4 status on 25 Oct (dev health, #517, #223, gaps)	Lim	4
12	UC9 attribution: #32 and the backend design docs name Eric as implementer (PR #72); #463 gives UC9 to Liu Jin. Agree and update the matrix	Eric, Liu Jin	6
13	Report section numbers for every member's use case and design problem (rubric G8, checklist "names the report section")	Lim	6, 9–22
14	Choose the live demo change, its author and the approving code owner; open the PR before 16:42	Pan, Lim	30
15	Two Google buyer accounts (leader, joiner), their display names on screen, and the admin grant for the listings check	Yorik	32
16	SWE5001 ECS environment up for the k6 pre-run on 24 Oct and during §5; keep the k6 summary for slide 33	Pan, Eric	24, 33
17	Recount the test numbers for slide 23 on 25 Oct	Liu Jin	23
18	Reconcile frontend ADR 0004 and the EC2 operator guide with the post-merge rebuild in ci-frontend.yml and ci-backend.yml	Pan, Lim	25, 26
19	Add the media CloudFront dcsjrkmummxi7 (S3 blocbuy-media-dev-402178232802, WAF) to the deployment view: since #570 the demo listings load their covers from it	Pan	28

A Appendix — check commands

Every command is read-only. Run them off the shared screen, from a checkout of `dev` at the repository root.

A.1 Health and recent runs

```
curl -s https://d1b3np88z657cp.cloudfront.net/api/actuator/health          # expect "status":"UP"
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow ci-frontend.yml --branch dev --limit 3
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow ci-backend.yml --branch dev --limit 3
gh pr list --repo PPsyrius/iss-groupbuy-team8 --base dev --state open      # what could still merge before th
e freeze
```

B Appendix — backup slides (questions only)

Backups follow slide 36 in the HTML deck and the PDF. In the deck press **B** for the index (BX) and click a row, or open `#B6` directly.

ID	Slide	Answers
B1	Agile artefacts appendix	Where are the backlog, stories, sprint plans, reviews and retros?
B2	Sprint 2 and 3 burndowns	How did earlier sprints burn down?
B3	ADR index	Which decisions are recorded, and where?
B4	Database schema, all tables	Which tables does each context own?
B5	Expiry and refund sequence	How does an expired team become refunds?
B6	UC6 refund recovery: candidates and decisions	How do you never refund twice?
B7	Test inventory	How many tests of each kind, and where do they run?
B8	Workflows, required checks and timings	What runs when, how long, and how often is it green?
B9	Known gaps and limitations	What is not done yet in the pipeline and on dev?
B10	SWE5001 production topology	What would scaling out look like?
B11	Stripe Sandbox refund, local evidence (20 Sep)	Is the refund real, and where has it been proven?
B12	Scale-out and load results (SWE5001 ECS + local baseline)	How does it scale, and how fast is it?